INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY

INNOVATION & LEADERSHIP

www.isquareit.edu.in

DEPARTMENT OF INFORMATION TECHNOLOGY

LAB MANUAL

COURSE: LAB PRACTICE IV

COURSE CODE:414447

CLASS: BEIT (2019 PATTERN)

SEMESTER-VII

COURSE CORDINATOR: DR. JYOTI SURVE

Vision of the Institute:

To be a premier academic institution that fosters diversity, value-added education and research, leading to sustainable innovations and transforming learners into leaders.

Mission of the Institute:

· To strive for academic excellence, knowledge enhancement, and critical thinking capabilities by adopting innovative and dynamic teaching-learning pedagogies

· To enrich and leverage interactions and associations through Industry-Academia partnerships

· To groom students so as to make them lifelong learners by helping them imbibe professional, entrepreneurial and leadership qualities

· To embrace an environment that allows all stakeholders to benefit from the technology-enabled processes and systems

Vision of the Department:

To be recognized as an ingenious department that provides value added education with technology excellence to transform students into lifelong learners and ethical professionals.

Mission of the Department:

- To provide excellent academics with positive atmosphere in the domain of Information Technology.
- To impart new technologies with innovative and interactive teaching learning.
- To encourage students for higher studies, skilled professionals and future entrepreneur.
- To groom the student with ethical and social values.

Program Educational Objectives (PEOs):

- PEO1: Possess strong fundamental concepts in mathematics, science, engineering and technology to be able to adapt to dynamic technological challenges and become effective professionals.

- PEO2: Acquire technical and analytical skills in the field of information technology so as to be able to analyse, design and implement efficient solutions to complex engineering problems with innovative approaches.

- PEO3: Develop proficiency in communication skills combined with domain specific competencies leading to success in higher education and as technologist and/or entrepreneur.

- PEO4: Be committed to abiding by ethical practices and become socially conscientious individuals

Program Outcomes (POs)

PO1: An ability to apply knowledge of mathematics, computing, science, engineering and technology.

PO2: An ability to define a problem and provide a systematic solution with the help of conducting experiments, analyzing the problem and interpreting the data.

PO3: An ability to design, implement, and evaluate a software or a software/hardware system, component, or process to meet desired needs within realistic constraints.

PO4: An ability to identify, formulates, and provides systematic solutions to complex engineering/Technology problems.

PO5: An ability to use the techniques, skills, and modern engineering technology tools, standard processes necessary for practice as a IT professional.

PO6: An ability to apply mathematical foundations, algorithmic principles, and computer science theory in the modeling and design of computer-based systems with necessary constraints and assumptions.

PO7: An ability to analyze and provide solution for the local and global impact of information technology on individuals, organizations and society.

PO8: An ability to understand professional, ethical, legal, security and social issues and responsibilities.

PO9: An ability to function effectively as an individual or as a team member to accomplish a desired goal(s).

PO10: An ability to communicate effectively in engineering community at large by means of effective presentations, report writing, paper publications, demonstrations.

PO11: An ability to understand engineering, management, financial aspects, performance, optimizations and time complexity necessary for professional practice.

PO12: An ability to engage in life-long learning and continuing professional development to cope up with fast changes in the technologies/tools with the help of electives, professional organizations and extracurricular activities.

Program Specific Outcomes (PSO)

PSO1: An ability to apply the theoretical concepts and practical knowledge of Information Technology in analysis, design, development and management of information processing systems and applications in the interdisciplinary domain.

PSO2: An ability to analyse a problem and identify and define the computing infrastructure and operations requirements appropriate to its solution. IT graduates should be able to work on large-scale computing systems.

PSO3: An understanding of professional, business and business processes, ethical, legal, security and social issues and responsibilities.

PSO4: Practice communication and decision-making skills through the use of appropriate technology and be ready for professional responsibilities

414447: Lab Practice IV

Teaching Scheme:	Credit Scheme:	Examination Scheme:
Practical (PR):02 hrs/week	01 credits	PR: 25 Marks TW: 25 Marks

COURSE EDUCATIONAL OBJECTIVES(CEO)

Course Objective	Description
CEO407.1	To be able to formulate deep learning problems corresponding to different applications.
CEO407.2	To be able to apply deep learning algorithms to solve problems of moderate complexity.
CEO407.3	To apply the algorithms to a real-world problem, optimize the models learned and report on the expected accuracy that can be achieved by applying the models

COURSE OUTCOME (CO)

Course Outcome	Description
CO407.1	Learn and Use various Deep Learning tools and packages.
CO407.2	Build and train a deep Neural Network models for use in various applications
CO407.3	Apply Deep Learning techniques like CNN, RNN Auto encoders to solve real word Problems
CO407.4	Evaluate the performance of the model build using Deep Learning

List of Practical Assignments

Sr. No.	Title of the Experiment/Assignment	CEO	CO	BTL
1	Study of Deep learning Packages: Tensorflow, Keras, Theano and PyTorch	CEO407.1,CEO407.2	CO407.1	BTL 2
2	Implementing Feedforward neural networks with Keras and TensorFlow	CEO407.2,CEO407.3	CO407.2, CO407.4	BTL 4
3	Build the Image classification model	CEO407.2,CEO407.3	CO407.3, CO407.4	BTL 4
4	Use Autoencoder to implement anomaly detection	CEO407.2,CEO407.3	CO407.2	BTL 4
5	Implement the Continuous Bag of Words (CBOW) Model	CEO407.2,CEO407.3	CO407.3	BTL 3
6	Object detection using Transfer Learning of CNN architectures	CEO407.2,CEO407.3	CO407.4	BTL 4

Assignment No.1

Aim: Study of Deep learning Packages: Tensorflow, Keras, Theano and PyTorch

Objectives:

1. Study of Deep learning Packages: Tensorflow, Keras, Theano and PyTorch.
2. Document the distinct features and functionality of the packages.

Theory:

1. Introduction to TENSORFLOW

TensorFlow is an end-to-end open-source platform for machine learning. TensorFlow is a rich system for managing all aspects of a machine learning system; however, this class focuses on using a particular TensorFlow API to develop and train machine learning models

TensorFlow APIs are arranged hierarchically, with the high-level APIs built on the low-level APIs. Machine learning researchers use the low-level APIs to create and explore new machine learning algorithms. In this class, you will use a high-level API named tf.keras to define and train machine learning models and to make predictions. tf.keras is the TensorFlow variant of the open-source Keras API.
The following figure shows the hierarchy of TensorFlow toolkits:

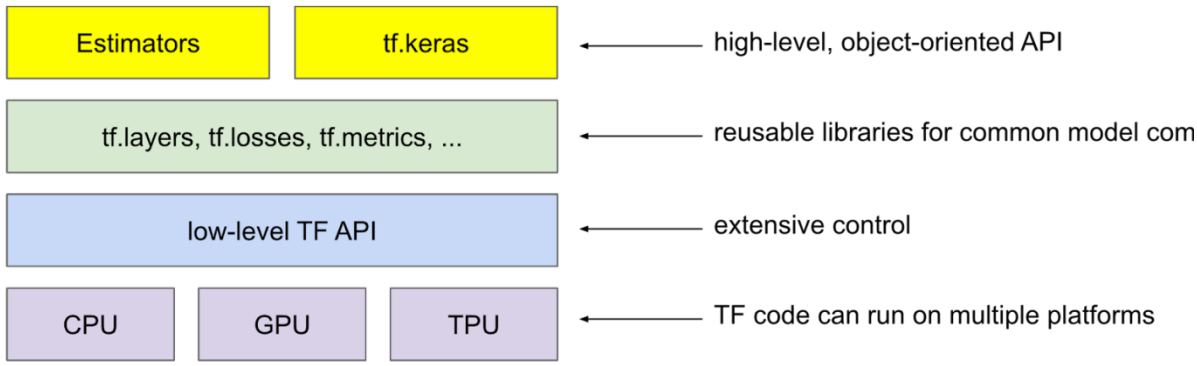
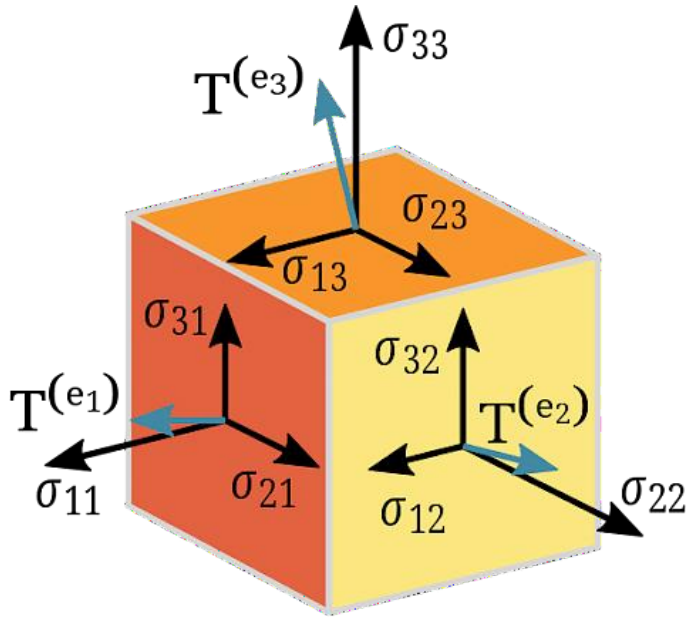


Figure 1. TensorFlow toolkit hierarchy.

Why is TensorFlow used in deep learning?
As the training of the models in deep learning takes extremely long because of the large amount of data, using TensorFlow **makes it much easier to write the code for GPUs or CPUs and then execute it in a distributed manner**
TensorFlow accepts data in the form of multi-dimensional arrays of higher dimensions called tensors. Multi-dimensional arrays are very handy in handling large amounts of data.
TensorFlow works on the basis of data flow graphs that have nodes and edges. As the execution mechanism is in the form of graphs, it is much easier to execute TensorFlow code in a distributed manner across a cluster of computers while using GPUs.
Tensor
Tensor forms the core framework of TensorFlow. All the computations in TensorFlow involve tensors. It is a matrix of n-dimensions that represents multiple types of data. A tensor can be the result of a computation or it can originate from the input data.



Graphs

Graphs describe all the operations that take place during the training. Each operation is called an op node and is connected to the other. The graph shows the op nodes and the connections between the nodes, but it does not display values.

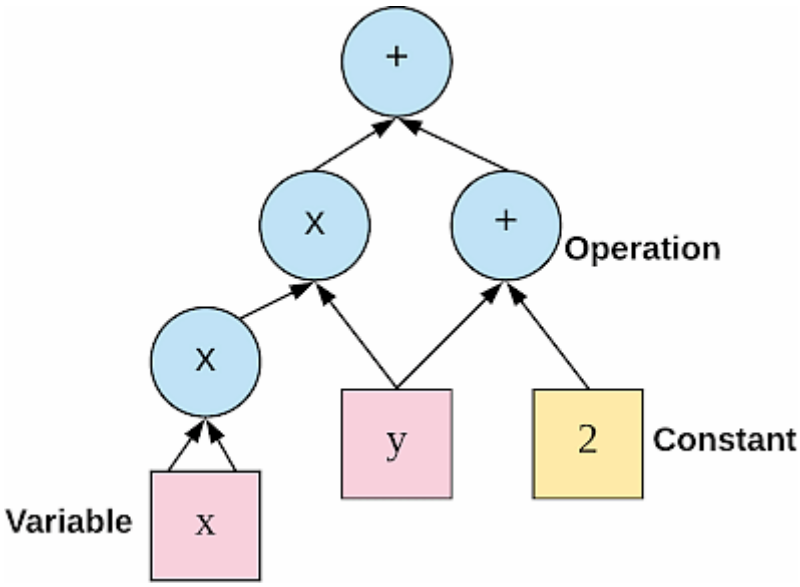
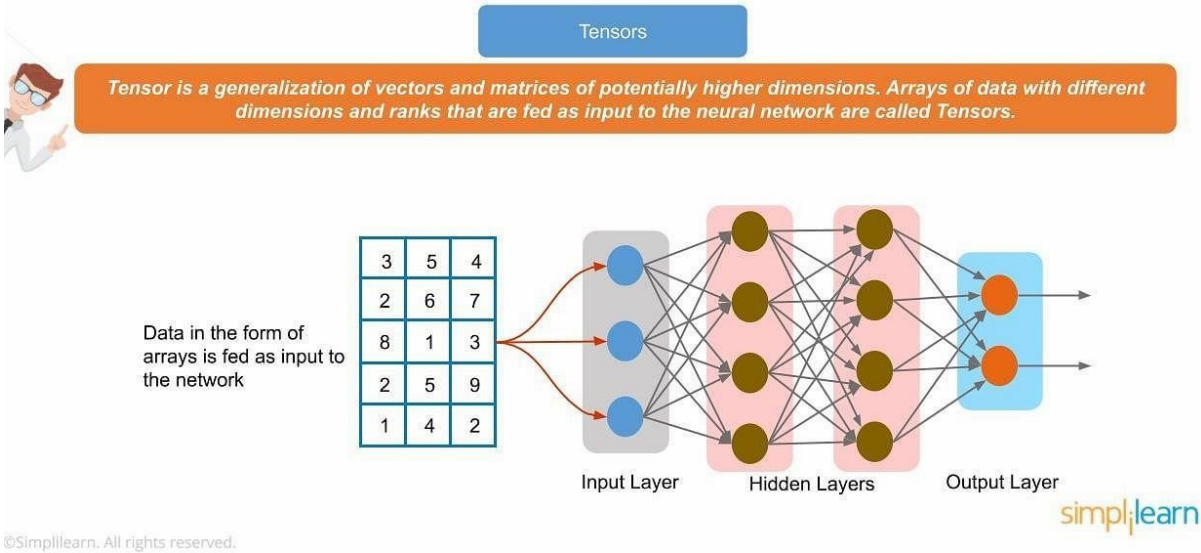


Fig: Graphs (source)

What are Tensors?

Tensor is a generalization of vectors and matrices of potentially higher dimensions. Arrays of data with varying dimensions and ranks that are fed as input to the neural network are called tensors.

For deep learning, especially in the training process, you will have large amounts of data that exist in a very complicated format. It helps when you are able to put, use, or store it in a compact way, which tensors provide, even if they appear in multi-dimensional arrays. When the data is stored in tensors and fed into the neural network, the output we get is as shown below:

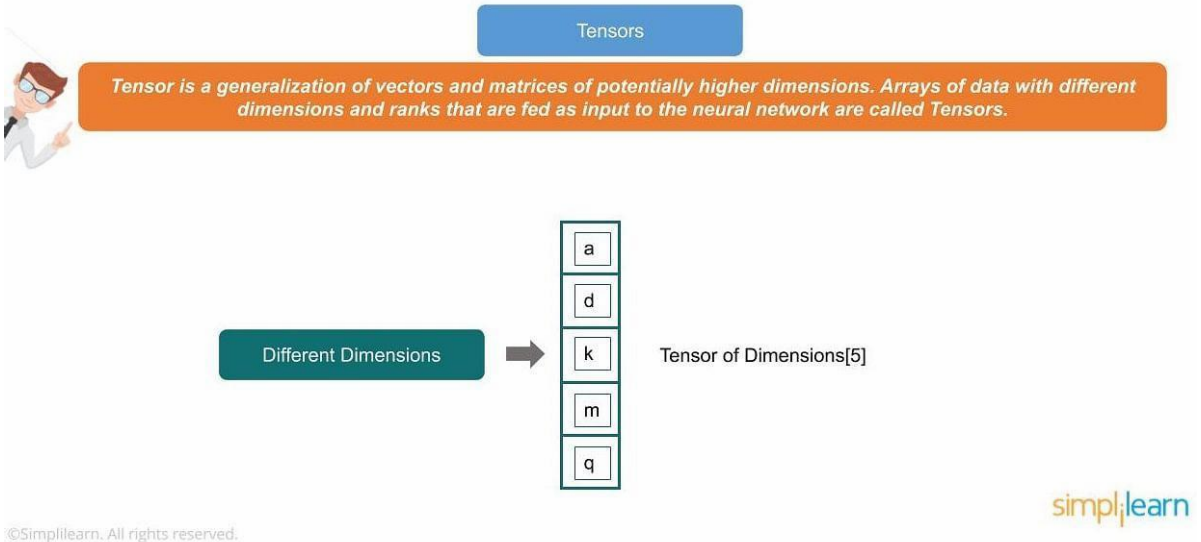


©Simplilearn. All rights reserved.

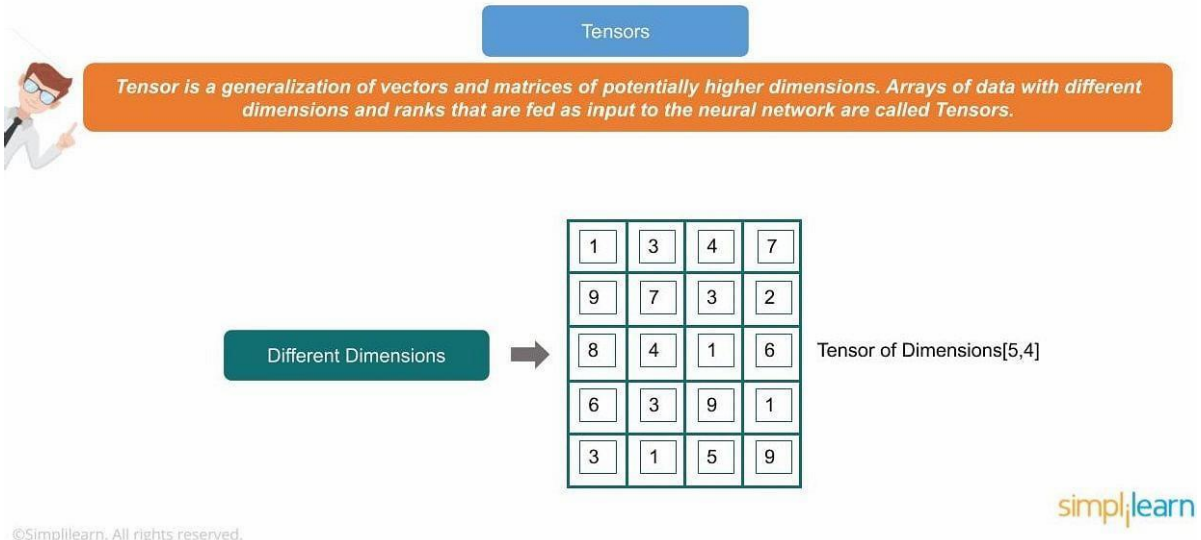
There are some terms associated with tensors that we need to familiarize ourselves with:

Dimension

Dimension is the size of the array elements. Below you can take a look at various types of dimensions:



©Simplilearn. All rights reserved.



©Simplilearn. All rights reserved.

Ranks

Tensor ranks are the number of dimensions used to represent the data. For example:

Rank 0 - When there is only one element. We also call this as a scalar.

Example: s = [2000]

Rank 1 - This basically refers to a one-dimensional array called a vector.
Example: v = [10, 11, 12]

2 Introduction to Keras

Keras is a high-level, deep learning API developed by Google for implementing neural networks. It is written in Python and is used to make the implementation of neural networks easy. It also supports multiple backend neural network computation.

Keras is relatively easy to learn and work with because it provides a python frontend with a high level of abstraction while having the option of multiple back-ends for computation purposes. Keras allows you to switch between different back ends. The frameworks supported by Keras are:

- Tensorflow
- Theano
- PlaidML
- MXNet
- CNTK (Microsoft Cognitive Toolkit)

Out of these five frameworks, TensorFlow has adopted Keras as its official high-level API. Keras is embedded in TensorFlow and can be used to perform deep learning fast as it provides inbuilt modules for all neural network computations.

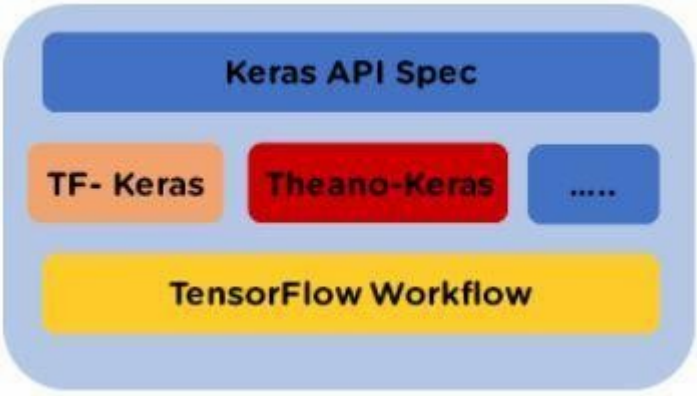


Figure 2: Keras Backend

Why Do We Need Keras?

- Keras is an API that was made to be easy to learn for people. Keras was made to be simple. It offers consistent & simple APIs, reduces the actions required to implement common code, and explains user error clearly.
- Prototyping time in Keras is less. This means that your ideas can be implemented and deployed in a shorter time. Keras also provides a variety of deployment options depending on user needs.

How to Build a Model in Keras?
The below diagram shows the basic steps involved in building a model in Keras:



Figure 3: Building a model

1. Define a network: In this step, you define the different layers in our model and the connections between them. Keras has two main types of models: Sequential and Functional models. You choose which type of model you want and then define the dataflow between them.
2. Compile a network: To compile code means to convert it in a form suitable for the machine to understand. In Keras, the model.compile() method performs this function. To compile the model, we define the loss function which calculates the losses in our model, the optimizer which reduces the loss, and the metrics which is used to find the accuracy of our model.
3. Fit the network: Using this, we fit our model to our data after compiling. This is used to train the model on our data.
4. Evaluate the network: After fitting our model, we need to evaluate the error in our model.

5. Make Predictions: We use model.predict() to make predictions using our model on new data.

- **Applications of Keras**

- Keras is used for creating deep models which can be productized on smartphones.
- Keras is also used for distributed training of deep learning models.
- Keras is used by companies such as Netflix, Yelp, Uber, etc.
- Keras is also extensively used in deep learning competitions to create and deploy working models, which are fast in a short amount of time.

3. Introduction to Theano

Theano is a Python library that lets you define mathematical expressions used in Machine Learning, optimize these expressions and evaluate those very efficiently by decisively using GPUs in critical areas. It can rival typical full C-implementations in most of the cases.

Theano was written at the LISA lab with the intention of providing rapid development of efficient machine learning algorithms. It is released under a BSD license.

Theano can be installed on Windows, MacOS, and Linux. The installation in all the cases is trivial. Before you install Theano, you must install its dependencies. The following is the list of dependencies −

- Python
- NumPy − Required
- SciPy − Required only for Sparse Matrix and special functions
- BLAS − Provides standard building blocks for performing basic vector and matrix operations

The optional packages that you may choose to install depending on your needs are −

- nose: To run Theano's test-suite
- Sphinx − For building documentation
- Graphiz and pydot − To handle graphics and images
- NVIDIA CUDA drivers − Required for GPU code generation/execution

Theano facilitates defining mathematical expressions used in ML development. Such expressions generally involve Matrix Arithmetic, Differentiation, Gradient Computation, and so on.

Theano first builds the entire Computational Graph for your model. It then compiles it into highly efficient code by applying several optimization techniques on the graph. The compiled code is injected into Theano runtime by a special operation called **function** available in Theano.

4. Introduction to Pytorch

PyTorch is defined as an open source machine learning library for Python. It is used for applications such as natural language processing. It is initially developed by Facebook artificial-intelligence research group, and Uber's Pyro software for probabilistic programming which is built on it.

Features

The major features of PyTorch are mentioned below −

Easy Interface − PyTorch offers easy to use API; hence it is considered to be very simple to operate and runs on Python. The code execution in this framework is quite easy.

Python usage − This library is considered to be Pythonic which smoothly integrates with the Python data science stack. Thus, it can leverage all the services and functionalities offered by the Python environment.

Computational graphs − PyTorch provides an excellent platform which offers dynamic computational graphs. Thus a user can change them during runtime. This is highly useful when a developer has no idea of how much memory is required for creating a neural network model.

PyTorch is known for having three levels of abstraction as given below −

- Tensor − Imperative n-dimensional array which runs on GPU.
- Variable − Node in computational graph. This stores data and gradient.
- Module − Neural network layer which will store state or learnable weights.

TensorFlow vs. PyTorch

We shall look into the major differences between TensorFlow and PyTorch below −

PyTorch	TensorFlow
PyTorch is closely related to the lua-based Torch framework which is actively used in Facebook.	TensorFlow is developed by Google Brain and actively used at Google.

PyTorch is relatively new compared to other competitive technologies.	TensorFlow is not new and is considered as a to-go tool by many researchers and industry professionals.
PyTorch includes everything in an imperative and dynamic manner.	TensorFlow includes static and dynamic graphs as a combination.
Computation graph in PyTorch is defined during runtime.	TensorFlow does not include any run time option.
PyTorch includes deployment features for mobile and embedded frameworks.	TensorFlow works better for embedded frameworks.

Algorithms/Pseudo code:

Sample Programs:

Kindly run it on Google colab

/* SAMPLE PROGRAM USING TENSORFLOW**/**

```
!pip install tensorflow
# import
import tensorflow as tf
import numpy as np
a = tf.constant(15)
b = tf.constant(20)
print(a+b)
# input
x = np.random.rand(100).astype(np.float32)
print(x)
# output - observed
y = x * 0.2 + 0.2
# Weight
W = tf.Variable(tf.random.normal([1]))
# bias
b = tf.Variable(tf.zeros([1]))
# Create a function for MSE - mean squared error
def mse_loss():
  ypred = W * x + b
  loss = tf.reduce_mean(tf.square(ypred-y))
  return loss
# Optimizer
optimizer = tf.keras.optimizers.Adam()
# Iterations
for step in range(5000):
  optimizer.minimize(mse_loss,var_list=[W,b])
  if step % 500 == 0:
    print(step, W.numpy(), b.numpy())
```

/* SAMPLE PROGRAM USING KERAS**/**

```
# !pip install tensorflow
# Step 1 - Load the dataset
from numpy import loadtxt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
## INPUT Variables ##
# x1 - Number of times pregnant
# x2 - plasma glucose
# x3 - diastolic blood pressure
# x4 - Triceps skin fold thickness
# x5 - 2-hour serum insulin
# x6 - bmi
# x7 - diabetes pedigree function
# x8 - age (yrs)
## Output Variable ##
# Class Variable - 0 or 1

dataset = loadtxt('/content/pima-indians-diabetes.csv',delimiter=',')
dataset
# [:,:] - first : is range of rows and second : is columns
# [start:end] - begins at start, ends at end-1
x = dataset[:,0:8]
print(type(x))
print(x.shape)
y = dataset[:,8]
print(y)
# Step 2 - Creating or define the Keras Model
# Sequential Model
# Layer1 -> Layer2 -> Layer3

model = Sequential()
# The model expects row of data with 8 variables
# 12 = nodes
model.add(Dense(12, input_shape=(8,), activation='relu'))
# Hidden Layer
# 8 = nodes
model.add(Dense(8, activation='relu'))
# Output layer
model.add(Dense(1,activation='sigmoid'))
# Step 3 - Compile the Keras model
# loss (error)
# optimizer (adam)
# metrics = accuracy
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
 Step 4 - Fit / Train the model
#1 = Epochs - number of iterations / passes
#2 - Batch - sample data
model.fit(x,y, epochs=150, batch_size=10)
# Step 5 - evaluate the model
model.evaluate(x,y)
```

/* SAMPLE PROGRAM USING THEANO**/**

```
# Theano
# Mathematical Operations
# Arrays - Multi-dimensional
# Installation
!pip install theano
import theano
from theano import *
import theano.tensor as T
import numpy as np
import pandas as pd
from theano import function

# scalar variables
v1 = T.dscalar()
v2 = T.scalar()
# subtraction
sres = v1-v2

#add
ares = v1+v2
 convert the results into functions
calcsres = theano.function([v1,v2],sres)
calcares = theano.function([v1,v2],ares)
calcares(12,23)
calcsres(13,12)
x = T.dmatrix('x')
y = T.dmatrix('y')

# addition
z = x+y
func = function([x,y],z)
m1 = [
   [1,2],
   [3,4]
]
m2 = [
   [4,5],
   [6,7]
]
func(m1,m2)
# element wise sum
# 00 -> [1+4] -> 5
```

/*** **SAMPLE** PROGRAM USING PYTORCH**/

```
# Two major objectives of PyTorch
#1. Replacement of Numpy to use the power of GPUs and other accelerators
#2. Automatic Differentiation Library helps in implementation of Neural Networks


# Feed Forward --> first stage
# Back Propagation --> second stage (differentiation) #
Tensors - Special Data Structures
# similar to an array, matrices #
similar to Numpy ndarrays
import torch
import numpy as np
# Tensor Initialization
### multiple ways
### 1 - using data
data = [
    [1,2],
    [3,4]
]
x_data = torch.tensor(data)

print(type(x_data))
### 2 - using numpy array
np_array = np.array(data)
x_np = torch.from_numpy(np_array) print(x_np)
print(type(x_np))
### 3 - using another tensor x_ones
= torch.ones_like(x_data)
print("One Tensor: \n",x_ones)
x_rand = torch.rand_like(x_data,dtype=torch.float)
print(x_rand)
#### more ways to create tensors
shape = (2,3)
random_tensor = torch.rand(shape)
print(random_tensor)
print(type(random_tensor))
ones_tensor = torch.ones(shape)
print(ones_tensor)
print(type(ones_tensor)) zeros_tensor
= torch.zeros(shape)
print(zeros_tensor)
print(type(zeros_tensor))
tensor = torch.rand(3,4)
print(tensor) tensor.shape
tensor.dtype tensor.device
# Tensor Operations
if torch.cuda.is_available():
  tensor = tensor.to('cuda')
  print("Device tensor is stored in ", tensor.device) #
Indexing, Slicing
tensor = torch.ones(4,4)
```

```
print(tensor)
print(tensor)
tensor1 = torch.zeros(4,4)
print(tensor1)
tensor2 = torch.cat([tensor,tensor1]) print(tensor2)
# Multiply Operation
tensor.mul(tensor1)
tensor * tensor1 tensor.T
# inplace - change the original tensor
tensor.add_(3)
print(tensor)
# from tensor to numpy t
= torch.ones(5)  print(t)
n = t.numpy()
print(n)
print(type(n))
```

Conclusion:

Thus we have studied different packages like tensor flow keras, pytorch, theano and their associated libraries

FAQ:

1. What is tensor?
2. What is dimension and Ranks in tensor flow?
3. How to build model in keras?
4. What is function operation in Theano?
5. How Pytorch is different from Tensor flow?

Output:

Assignment No.2

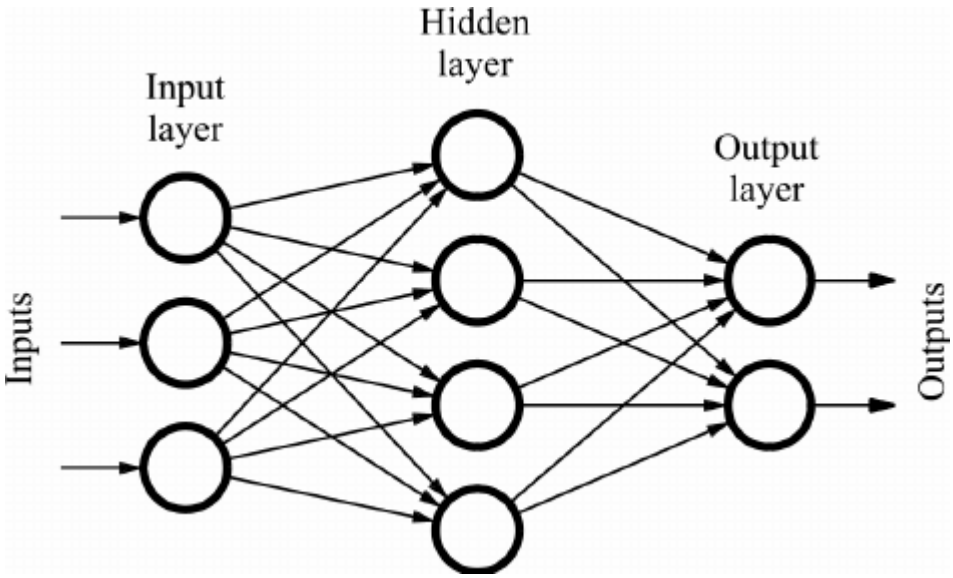
Aim: Implement Feed forward neural networks with Keras and TensorFlow

Objectives:

1. Study of deep learning Packages: Tensorflow, Keras.
2. Perform following operations on given dataset
    - **a.** Import the necessary packages
    - **b.** Load the training and testing data (MNIST/CIFAR10)
    - **c.** Define the network architecture using Keras
    - **d.** Train the model using SGD
    - **e.** Evaluate the network
    - **f.** Plot the training loss and accuracy

Theory:

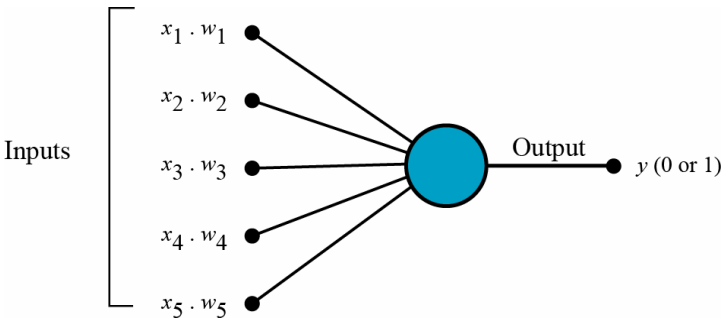
A Feed Forward Neural Network is an artificial neural network in which the connections between nodes does not form a cycle. The opposite of a feed forward neural network is a recurrent neural network, in which certain pathways are cycled. The feed forward model is the simplest form of neural network as information is only processed in one direction. While the data may pass through multiple hidden nodes, it always moves in one direction and never backwards.



How does a Feed Forward Neural Network work?

A Feed Forward Neural Network is commonly seen in its simplest form as a single layer perceptron. In this model, a series of inputs enter the layer and are multiplied by the weights. Each value is then added together to get a sum of the weighted input values. If the sum of the values is above a specific threshold, usually set at zero, the value produced is often 1, whereas if the sum falls below the threshold, the output value is -1. The single layer perceptron is an important model of feed forward neural networks and is often used in classification tasks. Furthermore, single layer perceptrons can incorporate aspects of machine learning. Using a property known as the delta rule, the neural network can compare the outputs of its nodes with the intended values, thus

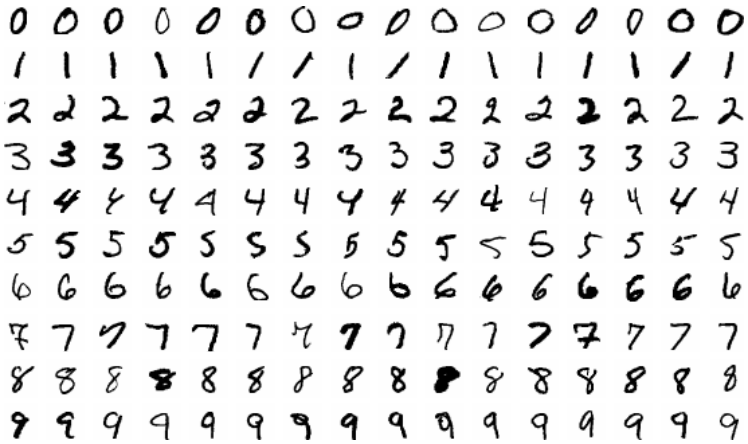
allowing the network to adjust its weights through training in order to produce more accurate output values. This process of training and learning produces a form of a gradient descent. In multi-layered perceptrons, the process of updating weights is nearly analogous, however the process is defined more specifically as back-propagation. In such cases, each hidden layer within the network is adjusted according to the output values produced by the final layer.



Load the training and testing data (MNIST/CIFAR10)

1. MNIST

Use the *full* MNIST dataset, consisting of 70,000 data points (7,000 examples per digit). Each data point is represented by a 784-d vector, corresponding to the (flattened) *28×28* images in the MNIST dataset. Our goal is to train a neural network (using Keras) to obtain > *90%* accuracy on this dataset.



MNIST (Modified National Institute of Standards and Technology) is a well-known dataset used in Computer Vision that was built by Yann Le Cun et. al. It is composed of images that are **handwritten digits (0-9).**

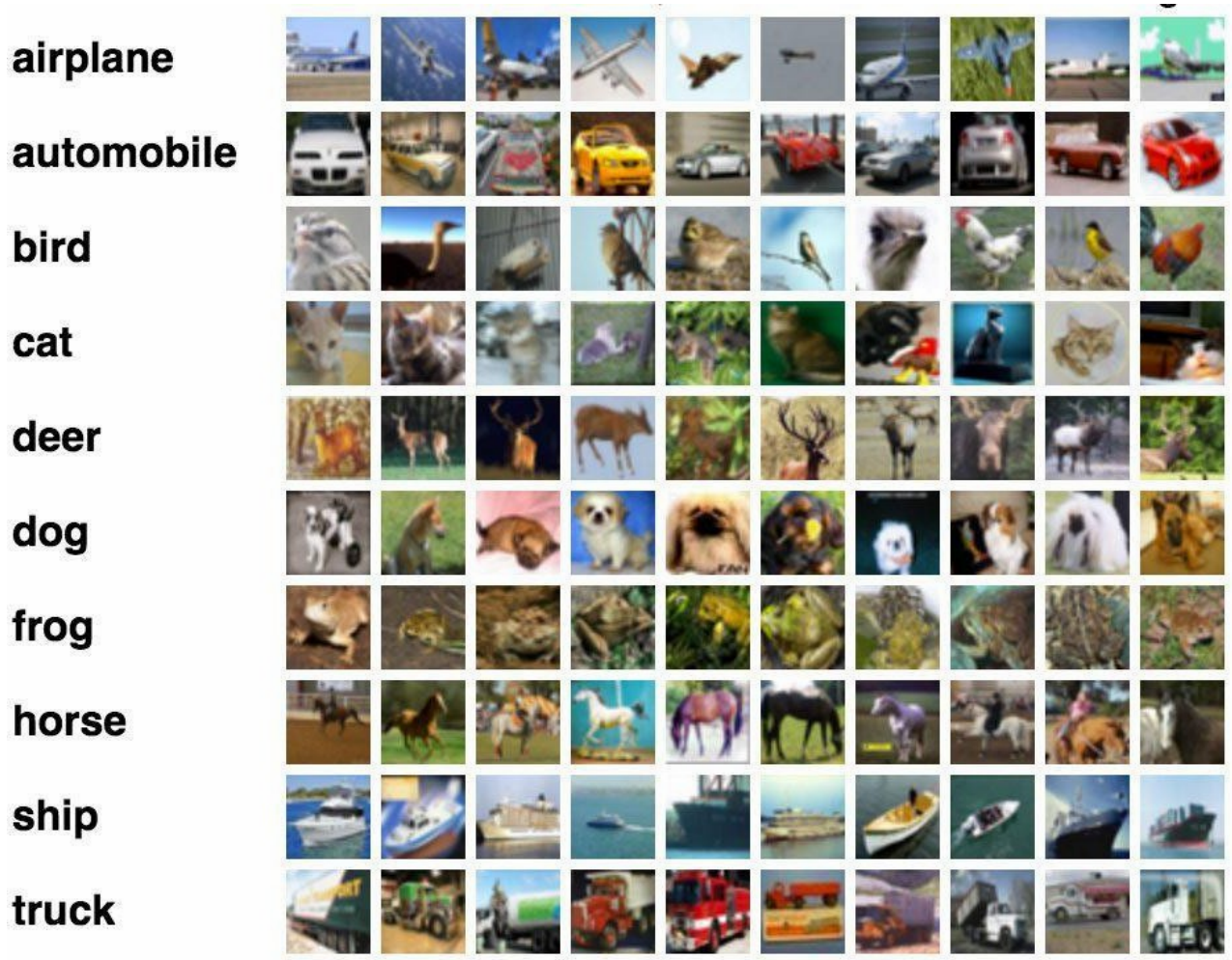
you can download it from below URL

http://yann.lecun.com/exdb/mnist/

2. CIFAR-10

When it comes to computer vision and machine learning, the MNIST dataset is the classic definition of a ―benchmark‖ dataset, one that is too easy to obtain high accuracy results on, and not representative of the images we'll see in the real world.

For a more challenging benchmark dataset, we commonly use CIFAR-10, a collection of 60,000, 32×32 RGB images, thus implying that each image in the dataset is represented by 32×32×3 = 3,072 integers. As the name suggests, CIFAR-10 consists of 10 classes, including airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. A sample of the CIFAR-10 dataset for each class can be seen in **Figure 2**.



Each class is evenly represented with 6,000 images per class. When training and evaluating a machine learning model on CIFAR-10, it's typical to use the predefined data splits by the authors and use 50,000 images for training and 10,000 for testing

As we'll find out, using Keras to build our network architecture is *substantially easier* than our pure Python version. In fact, the actual network architecture will only occupy *four lines of code* — the rest of the code in this example simply involves loading the data from disk, transforming the class labels, and then displaying the results.

Algorithm/Pseudo code:

Algorithm steps

1 Import the necessary packages

```
import tensorflow as tf
from keras.models import Sequential
from keras.datasets import mnist
import matplotlib.pyplot as plt
import numpy as np
import random
```

Activate Windows
Go to Settings to activate Windows.

Algorithm steps

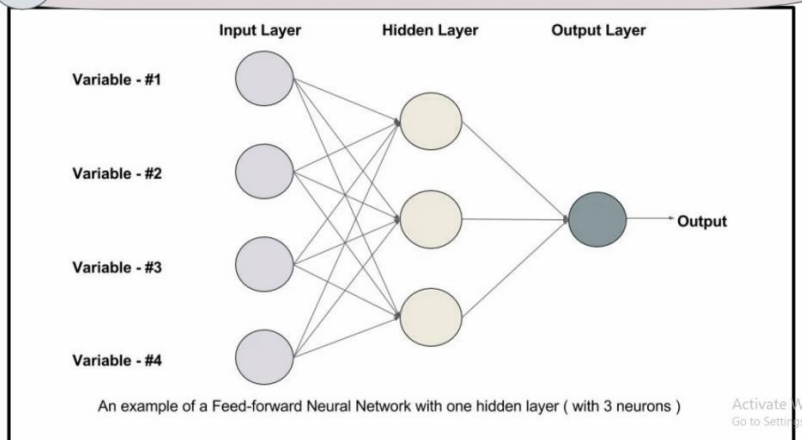
2 Load the training and testing data(MNIST/CIFAR10)

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train / 255
x_test = x_test / 255
```

Activate Windows
Go to Settings to activate Windows.

Algorithm steps

3 Define the network architecture using Keras



Activate Windows
Go to Settings to activate Windows.

Algorithm steps

Press Esc to exit full screen

4 Train the model using SGD

```
model.compile(optimizer = 'sgd', loss = 'sparse_categorical_crossentropy',
metrics = ["accuracy"])

H=model.fit(x_train, y_train,validation_data=(x_test,y_test),epochs = 5)
```

Activate Windows
Go to Settings to activate Windows.

Algorithm steps

Press Esc to exit full screen

4 Train the model using SGD

```
Epoch 1/5
1875/1875 [==============================] - 6s 3ms/step - loss: 0.1523 - accuracy: 0.9579 - val_loss: 0.1505 - val_accuracy: 0.9574
Epoch 2/5
1875/1875 [==============================] - 5s 2ms/step - loss: 0.1443 - accuracy: 0.9602 - val_loss: 0.1436 - val_accuracy: 0.9588
Epoch 3/5
1875/1875 [==============================] - 4s 2ms/step - loss: 0.1374 - accuracy: 0.9627 - val_loss: 0.1385 - val_accuracy: 0.9587
Epoch 4/5
1875/1875 [==============================] - 4s 2ms/step - loss: 0.1309 - accuracy: 0.9639 - val_loss: 0.1348 - val_accuracy: 0.9611
Epoch 5/5
1875/1875 [==============================] - 4s 2ms/step - loss: 0.1250 - accuracy: 0.9659 - val_loss: 0.1274 - val_accuracy: 0.9628
```

Algorithm steps

Press Esc to exit full screen

5 Evaluate the network

```
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Loss = %.3f" % test_loss)
print("Accuracy = %.3f" % test_acc)
```

```
n = random.randint(0, 9999)
plt.imshow(x_test[n])
plt.show()
prediction = model.predict(x_test)
print("The handwritten number in the image is %d" % np.argmax(prediction[n]))
```

Activate Windows
Go to Settings to activate Windows.



/******* SAMPLE PROGRAM ********/

```
# import the necessary packages
from sklearn.preprocessing import LabelBinarizer
from sklearn.metrics import classification_report
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.datasets import mnist
from tensorflow.keras import backend as K
import matplotlib.pyplot as plt
import numpy as np
import argparse


# construct the argument parse and parse the arguments
ap = argparse.ArgumentParser()
ap.add_argument("-o", "--output", required=True,
help="path to the output loss/accuracy plot")
args = vars(ap.parse_args())
# grab the MNIST dataset (if this is your first time using this
# dataset then the 11MB download may take a minute)
print("[INFO] accessing MNIST...")
((trainX, trainY), (testX, testY)) = mnist.load_data()
# each image in the MNIST dataset is represented as a 28x28x1
# image, but in order to apply a standard neural network we must
# first "flatten" the image to be simple list of 28x28=784 pixels
trainX = trainX.reshape((trainX.shape[0], 28 * 28 * 1))
testX = testX.reshape((testX.shape[0], 28 * 28 * 1))
# scale data to the range of [0, 1]
trainX = trainX.astype("float32") / 255.0
testX = testX.astype("float32") / 255.0
```

```
# convert the labels from integers to vectors
lb = LabelBinarizer()
trainY = lb.fit_transform(trainY)
testY = lb.transform(testY)
```

consider the label `3` and we wish to binarize/one-hot encode it — the label `3` now becomes:

`[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]` binarized:

Here is a second example, this time with the label `1`

`[0, 1, 0, 0, 0, 0, 0, 0, 0, 0]`

The second entry in the vector is set to one (since the first entry corresponds to the label `0`, while all other entries are set to zero.

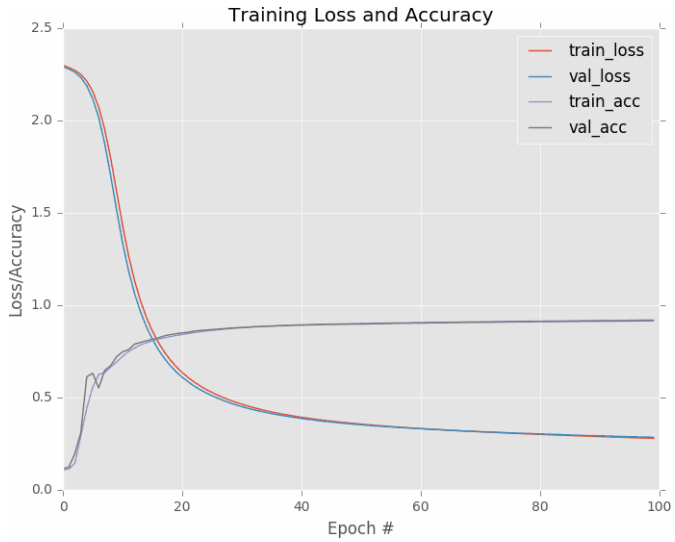
included the one-hot encoding representations for each digit, *0−9*, in the

```
0: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
1: [0, 1, 0, 0, 0, 0, 0, 0, 0, 0]
2: [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
3: [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
4: [0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
5: [0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
6: [0, 0, 0, 0, 0, 0, 1, 0, 0, 0]
7: [0, 0, 0, 0, 0, 0, 0, 1, 0, 0]
8: [0, 0, 0, 0, 0, 0, 0, 0, 1, 0]
9: [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
```

```
# define the 784-256-128-10 architecture using Keras
model = Sequential()
model.add(Dense(256, input_shape=(784,), activation="sigmoid"))
model.add(Dense(128, activation="sigmoid"))
model.add(Dense(10, activation="softmax"))
# train the model using SGD
print("[INFO] training network...")
sgd = SGD(0.01)
model.compile(loss="categorical_crossentropy",  optimizer=sgd,
metrics=["accuracy"])
H = model.fit(trainX, trainY, validation_data=(testX, testY),
epochs=100, batch_size=128)
# evaluate the network
print("[INFO] evaluating network...")
predictions = model.predict(testX, batch_size=128)
print(classification_report(testY.argmax(axis=1),
predictions.argmax(axis=1),
target_names=[str(x) for x in lb.classes_]))
# plot the training loss and accuracy
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, 100), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, 100), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 100), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 100), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
```

```
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
plt.savefig(args["output"])
$ python keras_mnist.py --output output/keras_mnist.png
[INFO] loading MNIST (full) dataset...
[INFO] training network...
Train on 52500 samples, validate on 17500 samples
Epoch 1/100
1s - loss: 2.2997 - acc: 0.1088 - val_loss: 2.2918 - val_acc: 0.1145
Epoch 2/100
1s - loss: 0.2792 - acc: 0.9204 - val_loss: 0.2844 - val_acc: 0.9160
[INFO] evaluating network...
precision recall f1-score support
0.0 0.94 0.96 0.95 1726
1.0 0.95 0.97 0.96 2004
2.0 0.91 0.89 0.90 1747
3.0 0.91 0.88 0.89 1828
4.0 0.91 0.93 0.92 1686

avg / total 0.92 0.92 0.92 17500
```



Conclusion:

Thus we have studied feed forward neural network architecture and its implementation using keras packages

FAQ:

1. What is neural network?
2. What is perceptron?
3. How to build model in keras?
4. What is feed forward means?
5. what is SGD?

Output:

Assignment No.3

Aim: Build the Image classification model

Objectives:

Build the Image classification model by dividing the model into following 4 stages:

a. Loading and preprocessing the image data

b. Defining the model‘s architecture

c. Training the model

d. Estimating the model‘s performance

Problem statement:

More than 25% of the entire revenue in E-Commerce is attributed to apparel , accessories. A major problem they face is categorizing these apparels from just the images especially when the categories provided by the brands are inconsistent. This poses an interesting computer vision problem that has caught the eyes of several deep learning researchers.

Fashion MNIST is a drop-in replacement for the very well known, machine learning hello world – MNIST dataset which can be checked out at _Identify the digits‘ practice problem. Instead of digits, the images show a type of apparel e.g. T-shirt, trousers, bag, etc.

LINK TO DOWNLOAD DATASET:

https://datahack.analyticsvidhya.com/contest/practice-problem-identify-

the-apparels/

Theory:

What is Image Classification?

Consider the below image:



You will have instantly recognized it – it‘s a (swanky) car. Take a step back and analyze how you came to this conclusion – you were shown an image and you classified the class it belonged to (a car, in this instance) There are potentially n number of categories in which a given image can be

classified. Manually checking and classifying images is a very tedious process.

The task becomes near impossible when we‘re faced with a massive number of images, say 10,000 or even 100,000. How useful would it be if we could automate this entire process and quickly label images per their corresponding class?

Self-driving cars are a great example to understand where image classification is used in the real-world. To enable autonomous driving, we can build an image classification model that recognizes various objects, such as vehicles, people, moving objects, etc. on the road.

Convolutional neural networks and image classification

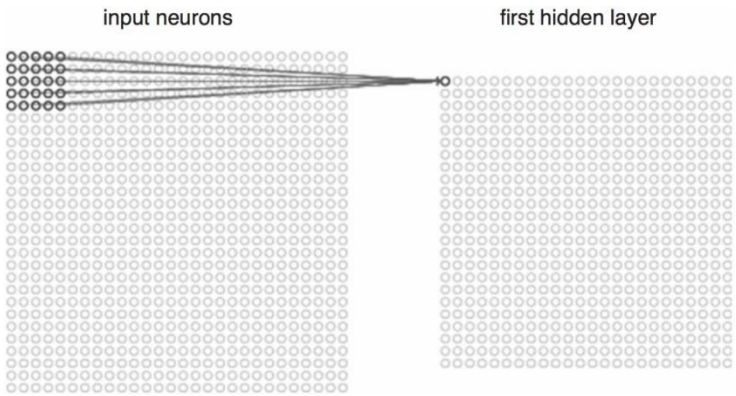
Convolutional neural networks (CNN) are a special architecture of artificial neural networks, proposed by Yann LeCun in 1988. CNN uses some features of the visual cortex. One of the most popular uses of this architecture is image classification. For example Facebook uses CNN for automatic tagging algorithms, Amazon — for generating product recommendations and Google — for search through among users‘ photos.

The main task of image classification is acceptance of the input image and the following definition of its class. This is a skill that people learn from their birth and are able to easily determine that the image in the picture is an elephant. But the computer sees the pictures quite differently:



Instead of the image, the computer sees an array of pixels. For example, if image size is 300 x 300. In this case, the size of the array will be 300x300x3. Where 300 is width, next 300 is height and 3 is RGB channel values. The computer is assigned a value from 0 to 255 to each of these numbers. This value describes the intensity of the pixel at each point.

The Convolution layer is always the first. The image (matrix with pixel values) is entered into it. Imagine that the reading of the input matrix begins at the top left of image. Next the software selects a smaller matrix there, which is called a **filter** (or neuron, or core). Then the filter produces convolution, i.e. moves along the input image. The filter‘s task is to multiply its values by the original pixel values. All these multiplications are summed up. One number is obtained in the end.



The network will consist of several convolutional networks mixed with nonlinear and pooling layers. When the image passes through one convolution layer, the output of the first layer becomes the input for the second layer. And this happens with every further convolutional layer. **The nonlinear layer** is added after each convolution operation. It has an activation function, which brings nonlinear property. The pooling layer follows the nonlinear layer. It works with width and height of the image and performs a downsampling operation on them. As a result the image volume is reduced
After completion of series of convolutional, nonlinear and pooling layers, it is necessary to attach **a fully connected layer**. This layer takes the output information from convolutional networks. Attaching a fully connected layer to the end of the network results in an N dimensional vector, where N is the amount of classes from which the model selects the desired class.

Algorithms/Pseudo codes

Setting up the Structure of our Image Data

- You should have 2 folders, one for the train set and the other for the test set. In the training set, you will have a .csv file and an image folder.
- The .csv file contains the names of all the training images and their corresponding true labels.
- The image folder has all the training images.
- The .csv file in our test set is different from the one present in the training set. This test set .csv file contains the names of all the test images.

Steps to Build our Model

- **Step 1-Create a new Python 3 notebook**
- install PyDrive. Now we will import a few required libraries:

```
!pip install PyDrive
import os
from pydrive.auth import GoogleAuth
from pydrive.drive import GoogleDrive
from google.colab import auth
from oauth2client.client  import GoogleCredentials
```

Step 2 : Import the libraries we'll need during our model building phase

```
import keras
```

```
from keras.models import Sequential
from keras.layers import Dense,Dropout, Flatten
from keras.layers import Conv2D, MaxPooling2D
from keras.utils import to_categorical
from keras.preprocessing import image
import numpy as np model building phase.
import pandas as pd
 import matplotlib.pyplot as plt
 from sklearn.model_selection import train_test_split
 from keras.utils import to_categorical
from tqdm import tqdm
```

Step 3: Recall the pre-processing steps we discussed earlier. We'll be using them here after loading the data.

```
Train = pd.read_csv('train.csv')
```

- Next, we will read all the training images, store them in a list, and finally convert that list into a numpy array.
- We have grayscale images, so while loading the images we will keep grayscale=True, if you have RGB images, you should set grayscale as False

```
train_image = []
for i in tqdm(range(train.shape[0])):
 img = image.load_
img('train/'+train['id'][i].astype('str')+'.png', target_size=(28,28,1), grayscale=True)
img = image.img_to_array(img)
 img = img/255
train_image.append(img)
 X = np.array(train_image)
```

Step 4: Creating a validation set from the training data.

- X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42, test_size=0.2)

Step 5: Define the model structure.

- This is another crucial step in our deep learning model building process. We have to define how our model will look and that requires answering questions like:

  How many convolution layers do we want?

  What should be the activation function for each layer?

  How many hidden units should each layer have?

- We will create a simple architecture with 2 convolutional layers, one dense hidden layer and an output layer.
- Next, we will compile the model we've created.

Step 6: Training the model.

- in this step, we will train the model on the training set images and validate it using, you guessed it, the validation set.

```
model.fit(X_train, y_train, epochs=10,validation_data=(X_test, y_test))
```

Step 7: Making predictions!

We‘ll initially follow the steps we performed when dealing with the training data. Load the test images and predict their classes using the *model.predict_classes()* function.

making predictions

prediction = model.predict_classes(test)

creating submission file

- sample = pd.read_csv('sample_submission_I5njJSF.csv')
- sample['label'] = prediction
- sample.to_csv('sample_cnn.csv', header=True,

index=False) Download this *sample_cnn.csv* file for you

result

Conclusion:

Thus we have successfully build image classification model on Fashion MNIST Dataset. FAQ:

1. How image classification works?
2. How we can use CNN in image classification?
3. What is Pydrive?
4. What is tqdm?
5. Explain various layers in keras, Dense,Flatten,Dropout

Output:

Assignment No.4

Aim: Use Auto encoder to implement anomaly detection

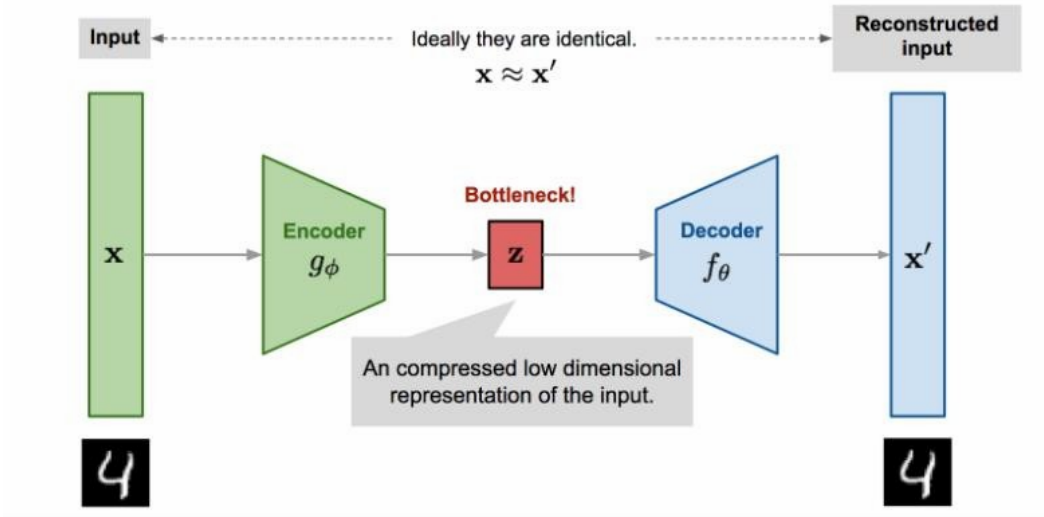
Objectives:

1. To study how auto encoders works
2. To perform following task as per sequence

a. Import required libraries
b. Upload / access the dataset
c. Encoder converts it into latent representation
d. Decoder networks convert it back to the original input
e. Compile the models with Optimizer, Loss, and Evaluation Metrics

Theory:

AutoEncoder is a generative unsupervised deep learning algorithm used for reconstructing high-dimensional input data using a neural network with a narrow bottleneck layer in the middle which contains the latent representation of the input data.



Autoencoder consists of an Encoder and a Decoder

- **Encoder network**: Accepts high-dimensional input data and translates it to latent low-dimensional data. The input size to an Encoder network is larger than its output size.
- **Decoder network**: The Decoder network receives the input from the Encoder coder's output. Decoder's objective is to reconstruct the input data. The output size of a Decoder network is larger than its input size.
- The Autoencoder accepts high-dimensional input data, compress it down to the latent-space representation in the bottleneck hidden layer; the Decoder takes the latent representation of the data as an input to reconstruct the original input data.

Autoencoders Usage

- **Dimensionality Reduction.** The Encoder encodes the input into the hidden layer to reduce the dimensionality of linear and nonlinear data; hence it is more powerful than PCA.
- **Recommendation Engines**
- **Anomaly Detection**: Autoencoders tries to minimize the reconstruction error as part of its training. Anomalies are detected by checking the magnitude of the reconstruction loss.

Denoising Images: An image that is corrupted can be restored to its original version.

- **Image recognition**: Stacked autoencoder are used for image recognition by learning the different features of an image.
- **Image generation**: Variational Autoencoder(VAE), a type of autoencoders, is used to generate images.

Algorithm/Pseudo codes

Anomaly detection using Auto encoders

We will train an Autoencoder Neural Network (implemented in Keras) in unsupervised (or semi-supervised) fashion for Anomaly Detection in credit card transaction data. The trained model will be evaluated on pre-labeled and anonymized dataset.

Follow the following steps to detect anomalies in a high dimension dataset. You can apply this to unbalanced datasets too.

- During the training, input only normal transactions to the Encoder. The bottleneck layer will learn the latent representation of the normal input data.
- The Decoder will use the bottleneck layers output to reconstruct the normal transactions of the original input data.
- A fraudulent transaction will be different from a normal transaction. The Autoencoder will have trouble reconstructing the fraudulent transaction, and hence the reconstruction error will be high.
- You can flag a new transaction is fraudulent based on a specified threshold value for the reconstruction error.

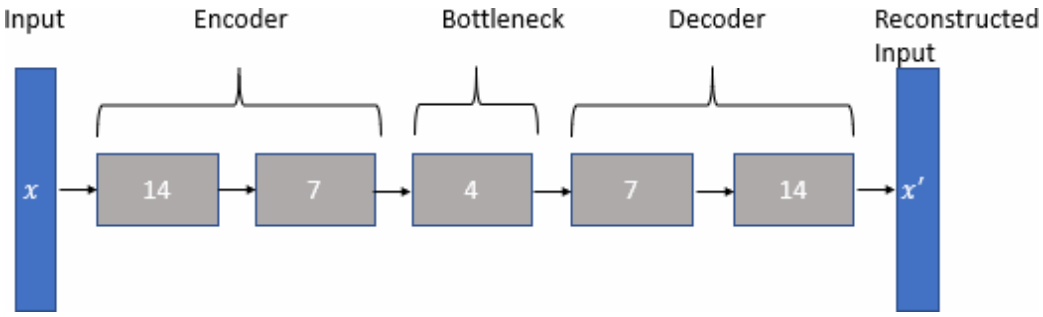
Dataset used here is Credit Card Fraud Detection

- https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud?resource=download

About dataset:

It contains data about credit card transactions that occurred during a period of two days, with 492 frauds out of 284,807 transactions. All variables in the dataset are numerical. The data has been transformed using PCA transformation(s) due to privacy reasons. The two features that haven't beenchanged are Time and Amount. Time contains the seconds elapsed between each transaction and the first transaction in the dataset.

1. **Create the Autoencoder**
   - The architecture of the autoencoder is shown below.



/*Sample code****/**

```
import pandas as pd import
numpy as np import
tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split


pip install tensorflow --user
!pip install keras
!pip install daytime
!pip install torch


from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, recall_score, accuracy_score,
precision_score
RANDOM_SEED = 2021
TEST_PCT = 0.3
LABELS = ["Normal","Fraud"]


dataset = pd.read_csv("E:\Teachning material\Deep learning BE IT 2019 course
\creditcard.csv") #dataset.head
print(list(dataset.columns))
dataset.describe()


#check for any  nullvalues
print("Any nulls in the dataset ",dataset.isnull().values.any() )
print('              ')
print("No. of unique labels ", len(dataset['Class'].unique()))
print("Label values ",dataset.Class.unique())
#0 is for normal credit card transaction
#1 is for fraudulent credit card transaction
print('              ')
print("Break down of the Normal and Fraud Transactions")
print(pd.value_counts(dataset['Class'], sort = True) )


#Visualizing the imbalanced dataset
count_classes = pd.value_counts(dataset['Class'], sort = True)
count_classes.plot(kind = 'bar', rot=0)
plt.xticks(range(len(dataset['Class'].unique())), dataset.Class.unique())
plt.title("Frequency by observation number")
plt.xlabel("Class") plt.ylabel("Number of
Observations");


# Save the normal and fradulent transactions in separate dataframe
```

```
normal_dataset = dataset[dataset.Class == 0]
fraud_dataset = dataset[dataset.Class == 1]
#Visualize transactionamounts for normal and fraudulent transactionsbins
= np.linspace(200, 2500, 100)
plt.hist(normal_dataset.Amount, bins=bins, alpha=1, density=True, label='Normal')
plt.hist(fraud_dataset.Amount, bins=bins, alpha=0.5, density=True, label='Fraud')
plt.legend(loc='upper right')
plt.title("Transaction amount vs Percentage of transactions")
plt.xlabel("Transaction  amount  (USD)") plt.ylabel("Percentage of
transactions");
plt.show()


#'''Time and Amount are the columns that are not scaled, so applying Standar
dScaler to only Amount and Time columns.
Normalizing the values between 0 and 1 did not work great for the dataset.'''
sc=StandardScaler()
dataset['Time'] = sc.fit_transform(dataset['Time'].values.reshape(-1, 1))
dataset['Amount'] = sc.fit_transform(dataset['Amount'].values.reshape(- 1, 1))


#'''The last column in the dataset is our target variable.'''

raw_data = dataset.values
# The last element contains if the transaction is normal which is represented by
a 0 and if fraud then 1
labels = raw_data[:, -1]
# The other data points are the electrocadriogram datadata
= raw_data[:, 0:-1]
train_data, test_data, train_labels, test_labels = train_test_split(data,
labels, test_size=0.2, random_state=2021


#'''Normalize the data to have a value between 0 and 1'''
min_val = tf.reduce_min(train_data)
max_val = tf.reduce_max(train_data)
train_data = (train_data - min_val) / (max_val - min_val)
test_data = (test_data - min_val) / (max_val - min_val)
train_data = tf.cast(train_data, tf.float32)
test_data = tf.cast(test_data, tf.float32)



#Use only normal transactions to train the Autoencoder.

Normal data has a value of 0 in the target variable. Using the target variable to
create a normal and fraud dataset.'''
train_labels = train_labels.astype(bool)
test_labels = test_labels.astype(bool)
```

```
normal_train_data = train_data[~train_labels]
normal_test_data = test_data[~test_labels]
fraud_train_data = train_data[train_labels]
fraud_test_data = test_data[test_labels]
print(" No. of records in Fraud Train Data=",len(fraud_train_data))
print(" No. of records in Normal Train data=",len(normal_train_data))
print(" No. of records in Fraud Test Data=",len(fraud_test_data)) print("
No. of records in Normal Test data=",len(normal_test_data))


nb_epoch = 50
batch_size = 64
input_dim = normal_train_data.shape[1] #num of columns, 30
encoding_dim = 14
hidden_dim_1 = int(encoding_dim / 2) #
hidden_dim_2=4
learning_rate = 1e-7


#input Layer
input_layer = tf.keras.layers.Input(shape=(input_dim, ))

#Encoder
encoder = tf.keras.layers.Dense(encoding_dim, activation="tanh",

activity_regularizer=tf.keras.regularizers.l2(learni
ng_rate))(input_layer)
encoder=tf.keras.layers.Dropout(0.2)(encoder)
encoder = tf.keras.layers.Dense(hidden_dim_1, activation='relu')(encoder) encoder
= tf.keras.layers.Dense(hidden_dim_2, activation=tf.nn.leaky_relu)(encoder)

# Decoder
decoder = tf.keras.layers.Dense(hidden_dim_1, activation='relu')(encoder)
decoder=tf.keras.layers.Dropout(0.2)(decoder)
decoder = tf.keras.layers.Dense(encoding_dim, activation='relu')(decoder)
decoder = tf.keras.layers.Dense(input_dim, activation='tanh')(decoder)

#Autoencoder
autoencoder = tf.keras.Model(inputs=input_layer, outputs=decoder)
autoencoder.summary()


#""Define the callbacks for checkpoints and early stopping"""
cp = tf.keras.callbacks.ModelCheckpoint(filepath="autoencoder_fraud.h5",
mode='min', monitor='val_loss', verbose=2, sa
ve_best_only=True)
# define our early stopping
early_stop = tf.keras.callbacks.EarlyStopping(
monitor='val_loss',
min_delta=0.0001,
patience=10, verbose=1,
```

```
mode='min',
restore_best_weights=True)


#Compile the Autoencoder


autoencoder.compile(metrics=['accuracy'],
loss='mean_squared_error',optimizer='adam')
#Train the Autoencoder
history = autoencoder.fit(normal_train_data, normal_train_data,
epochs=nb_epoch, batch_size=batch_size,
shuffle=True,
validation_data=(test_data, test_data),verbose=1,
callbacks=[cp, early_stop]
).history#Plot training and test

loss

plt.plot(history['loss'], linewidth=2, label='Train')
plt.plot(history['val_loss'], linewidth=2, label='Test')
plt.legend(loc='upper right')
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
#plt.ylim(ymin=0.70,ymax=1)
plt.show()


#"""Detect Anomalies on test data

Anomalies are data points where the reconstruction loss is higherTo

calculate the reconstruction loss on test data,
predict the test data and calculate the mean square error between the test data
and the reconstructed test data."""

test_x_predictions = autoencoder.predict(test_data)
mse = np.mean(np.power(test_data - test_x_predictions, 2), axis=1)
error_df = pd.DataFrame({'Reconstruction_error': mse,
'True_class': test_labels})


#Plotting the test data points and their respective reconstruction error sets a
threshold value to visualize
#if the threshold value needs to be adjusted.

threshold_fixed = 50
groups = error_df.groupby('True_class')fig,
ax = plt.subplots()
for name, group in groups:
```

```
ax.plot(group.index, group.Reconstruction_error, marker='o', ms=3.5, lin
estyle='',
label= "Fraud" if name == 1 else "Normal") ax.hlines(threshold_fixed,
ax.get_xlim()[0], ax.get_xlim()[1], colors="r", zorder=100, label='Threshold')
ax.legend()
plt.title("Reconstruction error for normal and fraud data")
plt.ylabel("Reconstruction error")
plt.xlabel("Data point index")
plt.show();


'''Detect anomalies as points where the reconstruction loss is greater thana
fixed threshold.
Here we see that a value of 52 for the threshold will be good.

Evaluating the performance of the anomaly detection'''

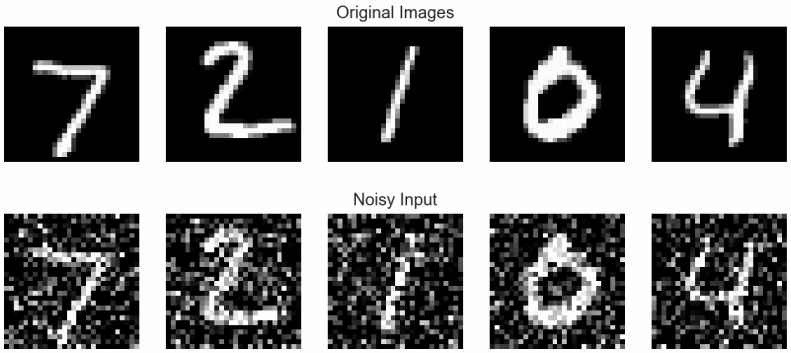
threshold_fixed =52
pred_y = [1 if e > threshold_fixed else 0 for e in error_df.Reconstruction_e
rror.values]
error_df['pred'] =pred_y
conf_matrix = confusion_matrix(error_df.True_class, pred_y)
plt.figure(figsize=(4, 4))
sns.heatmap(conf_matrix, xticklabels=LABELS, yticklabels=LABELS, annot=True,
fmt="d");
plt.title("Confusion matrix")
plt.ylabel('True class')
plt.xlabel('Predicted class')
plt.show()
# print Accuracy, precision and recall
print(" Accuracy: ",accuracy_score(error_df['True_class'], error_df['pred'])
)
print(" Recall: ",recall_score(error_df['True_class'], error_df['pred'])) print("
Precision: ",precision_score(error_df['True_class'], error_df['pred'
]))
#'''As our dataset is highly imbalanced, we see a high accuracy but a low recall
and precision.

Things to further improve precision and recall would add more relevant features,
different architecture for autoencoder, different hyperparameters, or a different
algorithm.'''
```

Conclusion:

- Autoencoder can be used as an anomaly detection algorithm when we have an unbalanced dataset where we have a lot of good examples and only a few anomalies.
- Autoencoders are trained to minimize reconstruction error. When we train the autoencoders on normal data or good data, we can hypothesize that the anomalies will have higher reconstruction errors than the good or normal data.

Output:



Assignment No.5

Aim: Implement the Continuous Bag of Words (CBOW) Model

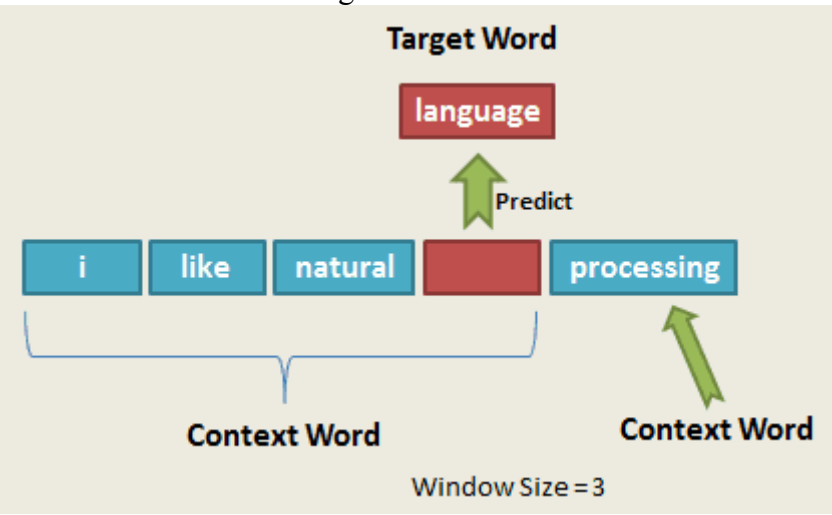
Objectives:

1. Study of Continuous Bag of Words (CBOW) Model
2. Stages can be:
   - **a.** Data preparation
   - **b.** Generate training data
   - **c.** Train model
   - **d.** Output

Theory:

What is the CBOW Model?

- The CBOW model tries to understand the context of the words and takes this as input. It then tries to predict words that are contextually accurate. Let us consider an example for understanding this.



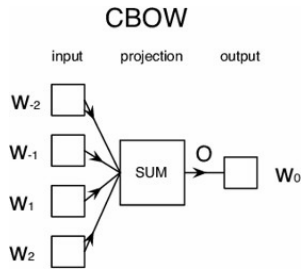
Consider the sentence:

'It is a pleasant day'

the word 'pleasant' goes as input to the neural network. We are trying to predict the word 'day' here.

We will use the one-hot encoding for the input words and measure the error rates with the one-hot encoded target word. Doing this will help us predict the output based on the word with least error.

The Model Architecture



Continuous Bag of Words (CBOW) single-word model:

- **Data Preparation:** Defining corpus by tokenizing text.
- **Generate Training Data:** Build vocabulary of words, one-hot encoding for words, word index.
- **Train Model:** Pass one hot encoded words through **forward pass**, calculate error rate by computing loss, and adjust weights using **back propagation**.
- **Output:** By using trained model calculate word vector and find similar words.

/*Sample code***/**

```
# Import the libraries from
numpy import array
from string import punctuationfrom
os import listdir
from collections import Counter from
nltk.corpus import stopwords
from keras.preprocessing.text import Tokenizerfrom
keras.models import Sequential
from keras.layers import Dense from
keras.layers import Dropoutfrom
pandas import DataFrame from
matplotlib import pyplot import nltk
nltk.download('stopwords')


# load doc into memory def
load_doc(filename):
# open the file as read onlyfile
= open(filename, 'r') # read all
text
text = file.read()#
close the file
file.close() return text
```

```
# turn a doc into clean tokensdef
clean_doc(doc):
# split into tokens by white spacetokens
= doc.split()
# remove punctuation from each token
table = str.maketrans('', '', punctuation) tokens =
[w.translate(table) for w in tokens]
# remove remaining tokens that are not alphabetic tokens =
[word for word in tokens if word.isalpha()]# filter out
stop words
stop_words  =  set(stopwords.words('english')) tokens =
[w for w in tokens if not w in stop_words]# filter out
short tokens
tokens = [word for word in tokens if len(word) > 1]return
tokens


# load doc and add to vocab
def add_doc_to_vocab(filename, vocab):#
load doc
doc = load_doc(filename)# clean
doc
tokens = clean_doc(doc)#
update counts
vocab.update(tokens)


# load doc, clean and return line of tokensdef
doc_to_line(filename, vocab):
# load the doc
doc = load_doc(filename)#
clean doc
tokens = clean_doc(doc)#
filter by vocab
tokens = [w for w in tokens if w in vocab]return
' '.join(tokens)

# load all docs in a directory
def process_docs(directory, vocab, is_trian):lines
= list()
# walk through all files in the folderfor
filename in listdir(directory):
# skip any reviews in the test set
if is_trian and filename.startswith('cv9'):
continue
if not is_trian and not filename.startswith('cv9'):
continue
# create the full path of the file to openpath =
directory + '/' + filename
# load and clean the doc
```

```
line = doc_to_line(path, vocab)# add
to list lines.append(line)
return lines

# load all docs in a directory
def process_docs1(directory, vocab):
# walk through all files in the folderfor
filename in listdir(directory):
# skip any reviews in the test setif
filename.startswith('cv9'):
continue
# create the full path of the file to openpath =
directory + '/' + filename
# add doc to vocab add_doc_to_vocab(path,
vocab)

# define vocab vocab =
Counter()
# add all docs to vocab
process_docs1('txt_sentoken/pos', vocab)
process_docs1('txt_sentoken/neg', vocab)#
print the size of the vocab print(len(vocab))
# print the top words in the vocab
print(vocab.most_common(50))

# evaluate a neural network model
def evaluate_mode(Xtrain, ytrain, Xtest, ytest):
scores = list()
n_repeats = 30
n_words = Xtest.shape[1] for i
in range(n_repeats):
# define network model =
Sequential()
model.add(Dense(50, input_shape=(n_words,), activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# compile network
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# fit network
model.fit(Xtrain, ytrain, epochs=50, verbose=2)#
evaluate
loss, acc = model.evaluate(Xtest, ytest, verbose=0)
scores.append(acc)
print('%d accuracy: %s' % ((i+1), acc))return
scores

# prepare bag of words encoding of docs
def prepare_data(train_docs, test_docs, mode):#
create the tokenizer
tokenizer = Tokenizer()
```

```
# fit the tokenizer on the documents
tokenizer.fit_on_texts(train_docs) #
encode training data set
Xtrain = tokenizer.texts_to_matrix(train_docs, mode=mode)#
encode training data set
Xtest = tokenizer.texts_to_matrix(test_docs, mode=mode)return
Xtrain, Xtest


# keep tokens with a min occurrence

min_occurane = 2
tokens = [k for k,c in vocab.items() if c >= min_occurane]
print(len(tokens))

# save list to file
def save_list(lines, filename):
# convert lines to a single blob of textdata =
'\n'.join(lines)
# open file
file = open(filename, 'w')# write
text file.write(data)
# close file
file.close()

# save tokens to a vocabulary file
save_list(tokens, 'vocab.txt')

# load the vocabulary vocab_filename
= 'vocab.txt' vocab =
load_doc(vocab_filename)vocab =
vocab.split()
vocab = set(vocab)
# load all training reviews
positive_lines  =  process_docs('txt_sentoken/pos',  vocab,  True)
negative_lines  =  process_docs('txt_sentoken/neg',  vocab,  True)
train_docs = negative_lines + positive_lines
# load all test reviews
positive_lines  =  process_docs('txt_sentoken/pos',  vocab,  False)
negative_lines  =  process_docs('txt_sentoken/neg',  vocab,  False)
test_docs = negative_lines + positive_lines
# prepare labels
ytrain = array([0 for _ in range(900)] + [1 for _ in range(900)])ytest
= array([0 for _ in range(100)] + [1 for _ in range(100)])

modes = ['binary', 'count', 'tfidf', 'freq']
results = DataFrame()
for mode in modes:
# prepare data for mode
Xtrain, Xtest = prepare_data(train_docs, test_docs, mode)#
evaluate model on data for mode
```

```
        results[mode] = evaluate_mode(Xtrain, ytrain,
        Xtest, ytest)# summarize results
print(results.describe())
# plot results
results.boxplot()
pyplot.show()

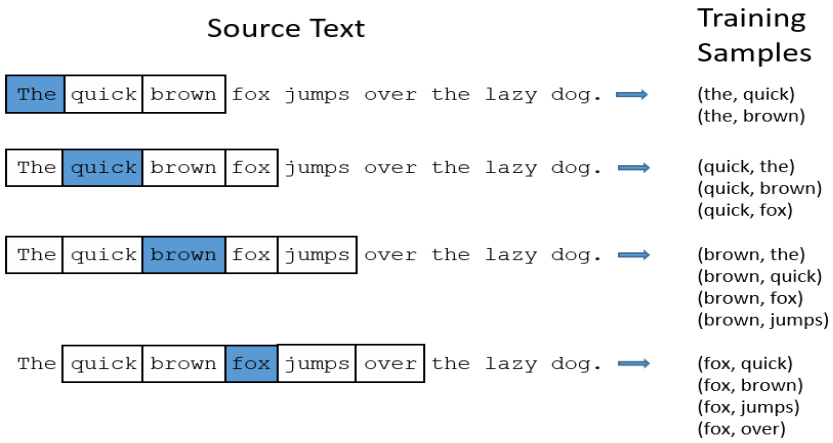
# classify a review as negative (0) or positive
(1) def predict_sentiment(review, vocab, tokenizer,
model):
# clean
tokens =
clean_doc(review)#
filter by vocab
tokens = [w for w in tokens if w in
vocab]# convert to line
line = '
'.join(tokens)#
encode
encoded = tokenizer.texts_to_matrix([line],
mode='freq')# prediction
yhat = model.predict(encoded,
verbose=0)return round(yhat[0,0])

# test  positive
text text = 'Best
movie ever!'
print(predict_sentiment(text, vocab, tokenizer,
model))# test negative text
text = 'This is a bad movie.'
print(predict_sentiment(text, vocab, tokenizer,
model))
```

Conclusion:

Thus we have studied working of CBOW model.

Ouput:



Assignment No.6

Aim:. To implement Object detection using Transfer Learning of CNN architectures

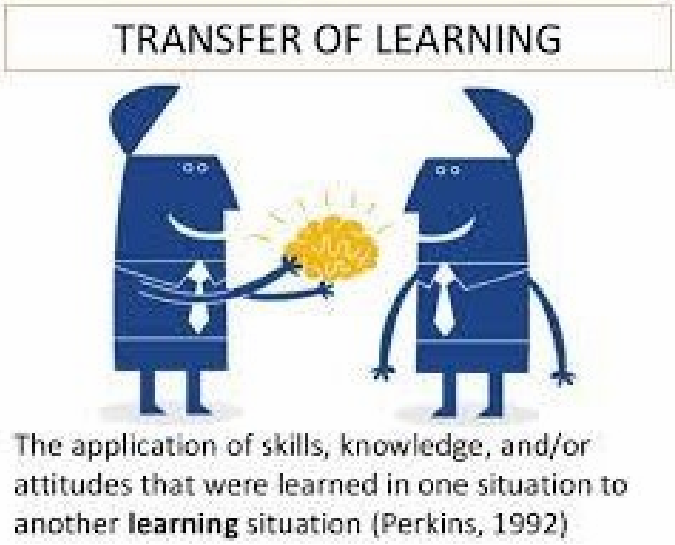
Objectives:

1. Study of different architectures of CNN.
2. Study of transfer learning and object detection.
3.perfom following tasks
    a. Load in a pre-trained CNN model trained on a large dataset
    b. Freeze parameters (weights) in model's lower convolutional layers
    c. Add custom classifier with several layers of trainable parameters to model
    d. Train classifier layers on training data available for task
    e. Fine-tune hyper parameters and unfreeze more layers as needed

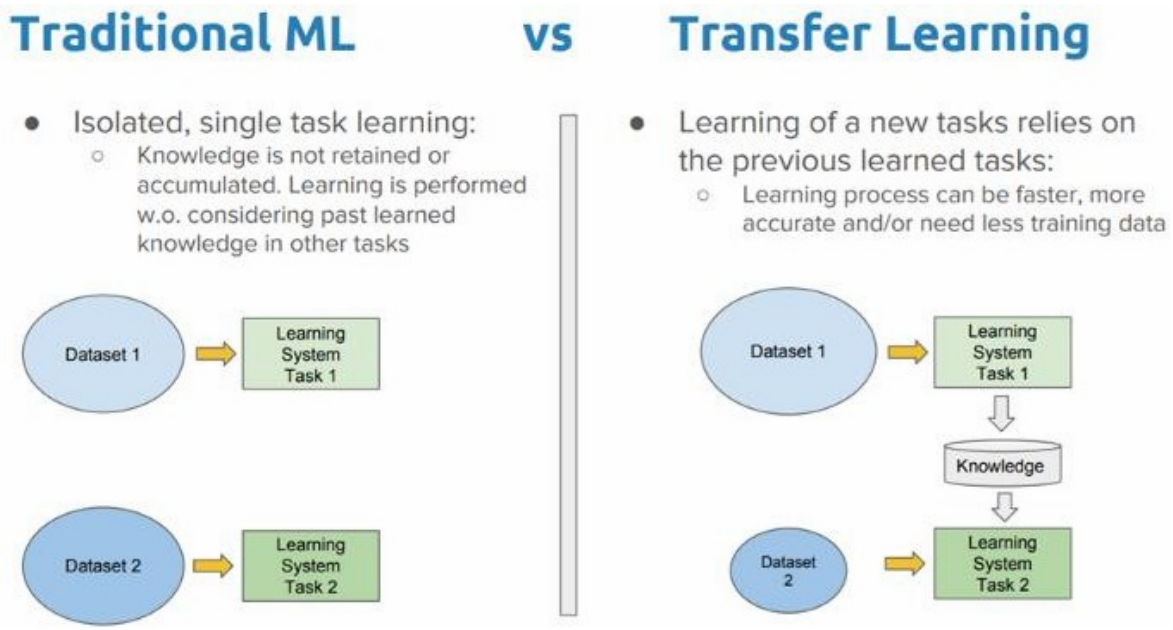
Theory:

What is transfer learning?

A teacher has years of experience in the particular topic he/she teaches. With all this accumulated information, the lectures that students get is a concise and brief overview of the topic. So it can be seen as a "transfer" of information from the learned to a novice.



A neural network is trained on a data. This network gains knowledge from this data, which is compiled as "weights" of the network. These weights can be extracted and then transferred to any other neural network. Instead of training the other neural network from scratch, we **"transfer"** the learned features.



What is a Pre-trained Model?

A pre-trained model is a model created by someone else to solve a similar problem. Instead of building a model from scratch to solve a similar problem, you use the model trained on other problem as a starting point.

Transfer learning is usually done for tasks where your dataset has too little data to train a full-scale model from scratch.

The most common incarnation of transfer learning in the context of deep learning is the following workflow:

1. Take layers from a previously trained model.
2. Freeze them, so as to avoid destroying any of the information they contain during future training rounds.
3. Add some new, trainable layers on top of the frozen layers. They will learn to turn the old features into predictions on a new dataset.
4. Train the new layers on your dataset.

A last, optional step is **fine-tuning**, which consists of unfreezing the entire model you obtained above (or part of it), and re-training it on the new data with a very low learning rate. This can potentially achieve meaningful improvements, by incrementally adapting the pretrained features to the new data.

Fine-tuning

Once your model has converged on the new data, you can try to unfreeze all or part of the base model and retrain the whole model end-to-end with a very low learning rate.

This is an optional last step that can potentially give you incremental improvements. It could also potentially lead to quick overfitting -- keep that in mind.

It is critical to only do this step *after* the model with frozen layers has been trained to convergence. If you mix randomly initialized trainable layers with trainable layers that hold pre-trained features, the randomly-initialized layers will cause very large gradient updates during training, which will destroy your pre-trained features.

It's also critical to use a very low learning rate at this stage, because you are training a much larger model than in the first round of training, on a dataset that is typically very small. As a result, you are at risk of overfitting very quickly if you apply large weight updates. Here, you only want to readapt the pretrained weights in an incremental way.

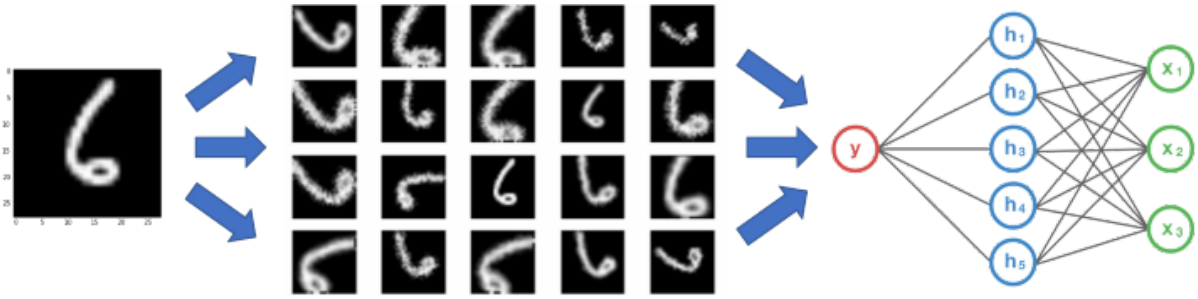
Data augmentation

Data augmentation is a process of artificially increasing the amount of data by generating new data points from existing data. This includes adding minor alterations to data or using machine learning models to generate new data points in the latent space of original data to amplify the dataset.

Minor changes such as flips or translations or rotations. Our neural network would think these are distinct images anyway.

A question may arise about the difference between augmented data and synthetic data.

- **Synthetic data**: When data is generated artificially without using real-world images. Synthetic data are often produced by Generative Adversarial Networks
- **Augmented data**: Derived from original images with some sort of minor geometric transformations (such as flipping, translation, rotation, or the addition of noise) in order to increase the diversity of the training set.



A convolutional neural network that can robustly classify objects even if its placed in different orientations is said to have the property called **invariance**. More specifically, a CNN can be invariant to **translation, viewpoint, size** or **illumination** (Or a combination of the above).

This essentially is the premise of **data augmentation**

Algorithms/steps

Transfer learning & fine-tuning

> Setup
> Introduction
> Freezing layers: understanding the trainable attribute
> Recursive setting of the trainable attribute
> The typical transfer-learning workflow
> Fine-tuning
> Transfer learning & fine-tuning with a custom training loop
> An end-to-end example: fine-tuning an image classification model on a cats vs. dogs datasetGetting the data
Standardizing the data
Using random data augmentation
> Build a model
> Train the top layer
> Do a round of fine-tuning of the entire model

Conclusion

Thus we have learned what is transfer learning, pretrained model and fine tuning

OUTPUT

